



移动智能终端生态联盟——大屏适配白皮书
Android T 无 androidX 实现悬停监听

编制单位：移动智能终端生态联盟·标准工作组

2025 年 5 月

针对应用暂时无法升级到 androidx 的应用，这里提供一种绕开 androidx 的方式实现悬停监听，对于这种数据库字段监听的建议作为临时过渡方案，最终还是希望切换为 androidx 的标准监听方法，标准监听方式参考： [《ITGSA 大屏设备应用适配白皮书》](#) 4.4.4 折叠屏状态监听和获取

demo 代码：

```
private static final Pattern FEATURE_PATTERN = Pattern.compile("[a-z]+-\\[(\\d+), (\\d+), (\\d+), (\\d+)]-?(flat|half-opened)?"); private static final String FEATURE_TYPE_FOLD = "fold"; private static final String FEATURE_TYPE_HINGE = "hinge"; private static final String PATTERN_STATE_FLAT = "flat"; private static final String PATTERN_STATE_HALF_OPENED = "half-opened";

private static final String DISPLAY_FEATURES = "display_features"; private final Uri mDisplayFeaturesUri = Settings.Global.getUriFor(DISPLAY_FEATURES); private ContentResolver mResolver; private ContentObserver mObserver;

@Override protected void onCreate(Bundle savedInstanceState) { super.onCreate(savedInstanceState);

mObserver = new SettingsObserver(); notifyFoldingFeature(); // onCreate 注册数据库监听

mResolver.registerContentObserver(mDisplayFeaturesUri, false /* notifyForDescendants */, mObserver /* ContentObserver */);

@Override public void onConfigurationChanged(@NonNull Configuration newConfig) { super.onConfigurationChanged(newConfig); notifyFoldingFeature(); }

@Override protected void onDestroy() { super.onDestroy(); // onDestroy 反注册数据库监听

mResolver.unregisterContentObserver(mObserver); }

private final class SettingsObserver extends ContentObserver {
    SettingsObserver() { super(new Handler(Looper.getMainLooper())); }
    @Override public void onChange(boolean selfChange, Uri uri) { if (mDisplayFeaturesUri.equals(uri)) { notifyFoldingFeature(); } }
}

// onCreate、onConfigurationChanged 和数据库变化 onChange 的时候调用 // 这里主要是获取数据库中折叠屏状态数据，并根据屏幕大小进行转换
private void notifyFoldingFeature() { WindowMetrics maxWindowMetrics = this.getWindowManager().getMaximumWindowMetrics(); Rect currentMaxWindowRect = maxWindowMetrics.getBounds(); CommonFoldingFeature foldingFeature = parseFromString(getFeatureString()); // 折叠态为 null if (foldingFeature == null) { return; } Rect foldingBounds = foldingFeature.getBounds(); // 由于数据库中只写 ROTATION 为 0 的值，因此这里需要在旋转屏幕的时候做转换 // rotation 为 0 是水平半折的机器 Rect foldingBounds = foldingFeature.getBounds(); if (foldingBounds.width() > foldingBounds.height()) { if (currentMaxWindowRect.width() != foldingBounds.width()) { int oriLeft = foldingBounds.left; int oriRight = foldingBounds.right; foldingBounds.left = foldingBounds.top; foldingBounds.top = oriLeft; foldingBounds.right = foldingBounds.bottom; foldingBounds.bottom = oriRight; } // rotation 为 0 是垂直半折的机器 } else { if (currentMaxWindowRect.height() != foldingBounds.height()) { int oriLeft = foldingBounds.left; int oriRight = foldingBounds.right; foldingBounds.left = foldingBounds.top; foldingBounds.top = oriLeft; foldingBounds.right = foldingBounds.bottom;
```

```

foldingBounds.bottom = oriRight;}}//当前叠屏状态数据, 建议应用可以保存上一次的数据。CommonFoldingFeature curFoldingFeature =
new CommonFoldingFeature(foldingBounds, foldingFeature.getType(), foldingFeature.getState());if (curFoldingFeature.getState()
== CommonFoldingFeature.STATE_FLAT) { //enterNormalMode() 展开态} else if (curFoldingFeature.getState() ==
CommonFoldingFeature.STATE_HALF_OPENED) {if (curFoldingFeature.getBounds().width() > curFoldingFeature.getBounds().height())
//enterTabletopMode() Orientation.HORIZONTAL 水平半折态} else { //enterBookMode() Orientation.VERTICAL 垂直半折态}} else
{}private String getFeatureString() {String settingsFeature = Settings.Global.getString(mResolver, DISPLAY_FEATURES);if
(TextUtils.isEmpty(settingsFeature)) {

settingsFeature = "";}return settingsFeature;}

//字符串解析 private CommonFoldingFeature parseFromString(@NonNull String string) {if (TextUtils.isEmpty(string)) {return
null;}Matcher featureMatcher = FEATURE_PATTERN.matcher(string);if (!featureMatcher.matches()) {throw new
IllegalArgumentException("Malformed feature description format: " + string);}try {String featureType = featureMatcher.group(1);

featureType = featureType == null ? "" : featureType;int type;switch (featureType) {case FEATURE_TYPE_FOLD:

type = CommonFoldingFeature.TYPE_FOLD;break;case FEATURE_TYPE_HINGE:

type = CommonFoldingFeature.TYPE_HINGE;break;default: {throw new IllegalArgumentException("Malformed feature type: " +
featureType);}}int left = Integer.parseInt(featureMatcher.group(2));int top = Integer.parseInt(featureMatcher.group(3));int
right = Integer.parseInt(featureMatcher.group(4));int bottom = Integer.parseInt(featureMatcher.group(5));Rect featureRect = new
Rect(left, top, right, bottom);if (isZero(featureRect)) {throw new IllegalArgumentException("Feature has empty bounds: " +
string);}String stateString = featureMatcher.group(6);

stateString = stateString == null ? "" : stateString;final int state;switch (stateString) {case PATTERN_STATE_FLAT:

state = CommonFoldingFeature.STATE_FLAT;break;case PATTERN_STATE_HALF_OPENED:

state = CommonFoldingFeature.STATE_HALF_OPENED;break;default:

state = -1;break;}return new CommonFoldingFeature(featureRect, type, state);catch (NumberFormatException e) {throw new
IllegalArgumentException("Malformed feature description: " + string, e);}}

private boolean isZero(@NonNull Rect rect) {return rect.height() == 0 && rect.width() == 0;}

//CommonFoldingFeature 为悬停监听封装的数据结构 private class CommonFoldingFeature {public static final int TYPE_FOLD = 1;public
static final int TYPE_HINGE = 2;public static final int STATE_FLAT = 1;public static final int STATE_HALF_OPENED = 2; //分割线位
置 private final Rect mBounds; //屏幕类型 fold 或者 hinge, 目前 findN 系列都是 fold private final int mType; //折叠状态 half_open 或者
flat 状态 private final int mState; public CommonFoldingFeature(Rect bounds, int type, int state) {

mBounds = new Rect(bounds);

mType = type;

mState = state;}public Rect getBounds() {return new Rect(mBounds);}public int getType() {return mType;}public int getState()
{return mState;}private String typeToString(int type) {switch (type) {case TYPE_FOLD: return "FOLD"; case TYPE_HINGE: return
"HINGE"; default: return "Unknown feature type (" + type + ")";}}private String stateToString(int state) {switch (state) {case
STATE_FLAT: return "FLAT"; case STATE_HALF_OPENED: return "HALF_OPENED"; default: return "Unknown feature state (" + state +
")";}}public String toString() {return "ExtensionDisplayFoldFeature { " + mBounds + ", type=" + typeToString(getType()) + ",
state=" + stateToString(mState) + " }";}}

```

Java

